

Anàlisi de Costos Computacionals Asimptòtics

Pràctica 1

Serafí Nebot Ginard

Universitat de les Illes Balears (UIB)

Grau en Enginyeria Informàtica

Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria presenta el desenvolupament d'una aplicació Java que permet visualitzar i comparar el cost computacional asimptòtic de quatre algorismes amb complexitats computacionals $O(n)$, $O(n \cdot \log n)$, $O(n^2)$ i $O(n^3)$. L'aplicació segueix estrictament el patró arquitectònic Model-Vista-Controlador (MVC), garantint una separació neta entre la lògica de negoci, la interfície gràfica i les dades. Es presenten els fonaments teòrics de l'anàlisi asimptòtic, el disseny del programari, la identificació de la instrucció crítica de cada algorisme, el càlcul de la constant multiplicativa i la predicció de temps d'execució per a mides d'entrada arbitràries.

Termes d'índex—anàlisi asimptòtic, complexitat computacional, Model-Vista-Controlador, Java Swing, SwingWorker, constant multiplicativa, concurrència

1. Introducció

L'anàlisi d'algorismes és una de les disciplines fonamentals de la informàtica. Permet avaluar l'eficiència d'un algorisme en funció de la mida de l'entrada, independentment del maquinari o del llenguatge de programació emprat. El concepte central és el cost asimptòtic, que descriu com creix el temps d'execució quan la mida de l'entrada n tendeix a infinit.

Quan es dissenya un algorisme, sovint existeixen múltiples estratègies per resoldre el mateix problema. L'anàlisi asimptòtic proporciona una eina objectiva per comparar-les: un algorisme amb cost $O(n \log n)$ serà sempre més eficient que un amb cost $O(n^2)$ per a entrades prou grans, independentment de les constants implicades. Aquesta observació, que pot semblar abstracta, té conseqüències pràctiques immediates: la diferència entre un algorisme lineal i un cúbic pot ser la diferència entre una execució de mil·lisegons i una de dies.

L'objectiu d'aquesta pràctica és doble. D'una banda, es proposa implementar i mesurar experimentalment el temps d'execució de quatre algorismes amb costos asimptòtics coneguts: lineal, quasilineal, quadràtic i cúbic. D'altra banda, l'aplicació s'ha de construir seguint el patró Model-Vista-Controlador (MVC) amb comunicació basada en el patró per esdeveniments, un principi de disseny que desacobla completament les responsabilitats de cada capa del programari.

El resultat és una aplicació d'escriptori Java Swing que permet a l'usuari seleccionar rangs de mida d'entrada, executar algorismes de forma individual o conjunta, visualitzar els resultats en una gràfica en temps real (amb escala lineal o logarítmica), consultar les constants multiplicatives

mitjanes i fer previsions de temps per a mides d'entrada no mesurades.

La memòria s'estructura de la següent manera: la Secció II descriu l'entorn de programació; la Secció III exposa els fonaments teòrics; la Secció IV detalla el disseny i la implementació de la pràctica; la Secció V analitza el càlcul del cost asimptòtic per la instrucció crítica; la Secció VI presenta els resultats; i la Secció VII tanca amb les conclusions.

2. Fonaments Teòrics

2.1. Anàlisi Asimptòtic de Costos

L'anàlisi asimptòtic permet classificar algorismes segons el seu comportament a llarg termini, és a dir, com creix el seu temps d'execució en funció de la mida de l'entrada [3]. Es defineixen les notacions següents:

- $O(f(n))$ (cota superior asimptòtica): existeixen constants $c > 0$ i n_0 tals que $T(n) \leq c \cdot f(n)$ per a tot $n \geq n_0$.
- $\Omega(f(n))$ (cota inferior): existeixen constants $c > 0$ i n_0 tals que $T(n) \geq c \cdot f(n)$ per a tot $n \geq n_0$.
- $\Theta(f(n))$ (cota ajustada): $T(n)$ és alhora $O(f(n))$ i $\Omega(f(n))$.

El temps d'execució real d'un algorisme es pot modelar com:

$$T(n) = c \cdot f(n) \quad (1)$$

on $f(n)$ és la funció de cost teòric i c és la *constant multiplicativa*, que depèn del maquinari, el compilador i altres factors constants. La notació asimptòtica extreu aquesta constant per a centrar-se només en l'ordre de creixement.

2.2. Ordres de Creixement

Les quatre funcions de cost que s'analitzen en aquesta pràctica presenten comportaments de creixement dràsticament diferents. La Taula 1 il·lustra aquesta diferència amb valors numèrics concrets.

Taula 1. COMPARACIÓ D'ORDRES DE CREIXEMENT PER A DIFERENTS VALORS DE n .

n	n	$n \log_2 n$	n^2	n^3
10	10	33	100	1000
100	100	664	10000	10^6
1000	1000	9966	10^6	10^9
10000	10000	132877	10^8	10^{12}

Com es pot observar, per a $n = 10000$, un algorisme cúbic realitza 10^{12} operacions bàsiques, mentre que un de lineal només en fa 10^4 : una diferència de vuit ordres de magnitud. Si cada operació triga 1 nanosegon, l'algorisme lineal acabaria en 10 microsegons, mentre que el cúbic necessitaria aproximadament 17 minuts.

2.3. Instrucció crítica

La *instrucció crítica* és aquella operació dins l'algorisme que s'executa el major nombre de vegades i que, per tant, domina el cost total [7]. Identificar-la és clau per determinar la complexitat asimptòtica. En algorismes iteratius, la instrucció crítica és habitualment l'operació dins el bucle més intern. El nombre total d'execucions d'aquesta instrucció depèn del nombre d'iteracions de tots els bucles que l'envolten.

Per exemple, en un algorisme amb dos bucles imbricats, cadascun iterant n vegades, la instrucció crítica s'executa $n \times n = n^2$ vegades, i per tant el cost asimptòtic és $O(n^2)$.

2.4. Patró Model-Vista-Controlador (MVC)

El patró MVC [1] divideix una aplicació en tres components amb responsabilitats ben definides:

- 1) **Model:** encapsula les dades i la lògica de negoci. Notifica als observadors quan el seu estat canvia, però no coneix la interfície gràfica. En el nostre cas, emmagatzema les mesures d'execució, calcula constants multiplicatives i fa prediccions.
- 2) **Vista:** és responsable de la presentació visual. Renderitza les dades del model i captura les accions de l'usuari, però no coneix la lògica de negoci. En el nostre cas, dibuixa la gràfica, mostra la taula de resultats i ofereix els controls.
- 3) **Controlador:** actua com a intermediari. Reb events semàntics de la vista, invoca operacions sobre el model i actualitza la vista amb resultats. En el nostre cas, gestiona els fils d'execució dels algorismes.

Un principi fonamental del MVC ben implementat és que la **vista mai no ha d'exposar els seus components**

visuals (botons, camps de text, etc.) al Controlador ni a cap altra capa. Tota comunicació ha de ser a través d'events semàntics i mètodes que operen amb dades. Aquesta restricció és crucial perquè, si el controlador accedís directament als botons o camps de text de la vista, qualsevol canvi a la interfície gràfica obligaria a modificar també el controlador, trencant el principi de separació de responsabilitats.

En implementacions ingènues del MVC, és comú veure patrons com `view.getButton().addActionListener(...)`, on el controlador obté directament un `JButton` de la vista. Això viola el patró perquè crea un acoblament directe entre el controlador i la representació visual concreta. En aquesta implementació, s'ha evitat completament aquest anti-patró.

2.5. Patró per Esdeveniments (Observer)

El patró Observer [2] estableix una relació un-a-molts entre objectes, de manera que quan un objecte (el *subjecte*) canvia d'estat, tots els seus observadors són notificats automàticament. En el context d'aquesta pràctica, s'utilitza en dues direccions:

- 1) **Model $\rightarrow$ Vista:** El model defineix una interfície `ModelListener` amb el mètode `modelChanged()`. Quan s'afegeix una mesura o s'esborren dades, el model invoca aquest mètode, i la vista actualitza la taula i la gràfica.
- 2) **Vista $\rightarrow$ Controlador:** La vista defineix una interfície `ViewListener` amb set mètodes semàntics (`onStartAlgorithm`, `onStopAlgorithm`, `onStartAll`, `onStopAll`, `onPredict`, `onClear`, `onLogScaleChanged`). El controlador implementa aquesta interfície i reb events ja processats, mai components Swing.

Aquesta doble aplicació del patró Observer garanteix un desacoblament complet: el model no sap qui l'observa, i el controlador no sap com es la interfície gràfica. Es podria substituir tota la vista Swing per una interfície web o de terminal sense modificar ni una sola línia del model o del controlador.

2.6. Concurrència i el Fil EDT

En Java Swing, totes les operacions sobre components visuals s'han de realitzar des del fil de despatx d'events (*Event Dispatch Thread*, EDT) [6]. Si un algorisme que consumeix molt de recursos s'executa directament al fil EDT, la interfície es bloquejaria fins que acabes, implicant qualsevol interacció amb l'usuari.

La solució estàndard és `SwingWorker`, una classe que permet executar tasques llargues en un fil de fons i publicar resultats parcials al fil EDT de forma segura. El cicle de vida d'un `SwingWorker` és:

- 1) `doInBackground()`: s'executa en un fil de fons. Aquí s'executa l'algorisme i es crida `publish()` per enviar resultats intermedis.

- 2) `process(List<V>)`: s'invoca al fil EDT amb els resultats publicats. Aquí s'afegeixen les mesures al model.
- 3) `done()`: s'invoca al fil EDT quan la tasca acaba o es cancel·la. Aquí s'actualitza l'estat visual del botó.

La cancel·lació es cooperativa: `SwingWorker.cancel(true)` envia una interrupció al fil de fons, que ha de comprovar periòdicament `Thread.currentThread().isInterrupted()` per aturar-se netament.

3. Disseny i Implementació de la Pràctica

3.1. Capa del Model

La capa del model consta de tres classes, cadascuna amb una responsabilitat clara i ben delimitada.

3.1.1. AlgorithmType (Enum). Defineix els quatre algorismes com a constants d'un enum. Cada constant implementa dos mètodes abstractes:

- `execute(long n)`: executa l'algorisme amb mida n i retorna un resultat numèricament dependent de l'entrada per evitar que el compilador JIT l'optimitzi o l'elimini.
- `theoreticalCost(long n)`: retorna el valor numèric de $f(n)$.

A més, cada algorisme té associat un nom visible, una notació matemàtica i un color per a la gràfica.

Per exemple, la constant de l'algorisme quadràtic retorna n^2 i el seu mètode `execute()` implementa dos bucles imbricats de n iteracions cadascun, on la instrucció crítica és `sum += i * j`. La cancel·lació es comprova al bucle extern per no penalitzar el rendiment.

Els mètodes `execute()` no contenen cap lògica de cancel·lació: són funcions pures que simplement executen els bucles i retornen un resultat. La responsabilitat de detectar la cancel·lació recau exclusivament al controlador, que comprova `isCancelled()` entre cada valor de n i entre cada repetició. Aquesta separació manté el model lliure de conceptes de concurrència i elimina la necessitat de comprovacions periòdiques amb `isInterrupted()` dins els bucles.

3.1.2. Measurement. Classe immutable que emmagatzema el resultat d'una mesura: el tipus d'algorisme, la mida n , el temps en nanosegons i la constant multiplicativa, que es pre-calcuła al constructor:

$$c = \frac{T(n)}{f(n)} \quad (2)$$

on $T(n)$ és el temps mesurat en nanosegons i $f(n)$ és el cost teòric retornat per `theoreticalCost(n)`.

Pre-calcular c a cada mesura individual permet després obtenir la mitjana de forma directa, sense necessitat de recalculer divisions repetidament.

3.1.3. Model. Classe central que manté un `EnumMap` de llistes de mesures. Els mètodes principals són:

- `addMeasurement(Measurement m)`: afegeix una mesura, ordena la llista per n creixent (per facilitar el dibuix de la gràfica) i notifica els observadors.
- `getAverageConstant(AlgorithmType t)`: retorna la mitjana aritmètica de les constants individuals.
- `predict(AlgorithmType t, long n)`: calcula el temps predit com $\hat{T}(n) = \bar{c} \cdot f(n)$, on $\bar{c}$ es la constant mitjana.
- `clearMeasurements(AlgorithmType t)`: esborra les mesures d'un algorisme concret i notifica els observadors.
- `clearAll()`: esborra totes les mesures i ho notifica.

Tots els mètodes que modifiquen o llegeixen l'estat són `synchronized`, ja que els fils de fons dels `SwingWorker` poden accedir al model concurrentment. El model implementa el patró `Observer` mitjançant la interfície `ModelListener`, que exposa un únic mètode `modelChanged()`. El model manté una llista d'observadors i els notifica tots cada vegada que s'afegeix o s'esborren mesures, invocant `modelChanged()` sobre cadascun.

3.2. Capa de la Vista

3.2.1. ViewListener (Interfície). Defineix les operacions permeses que possibiliten la comunicació entre la vista i el controlador. Els set mètodes semàntics que exposa són:

- `onStartAlgorithm(AlgorithmType, long[])`: inicia l'execució d'un algorisme concret per als valors de n indicats.
- `onStopAlgorithm(AlgorithmType)`: cancel·la l'execució d'un algorisme concret.
- `onStartAll(long[])`: inicia tots els algorismes simultàniament.
- `onStopAll()`: cancel·la tots els algorismes en execució.
- `onPredict(long)`: sol·licita una predicció de temps per a la mida n indicada.
- `onClear()`: esborra totes les mesures del model.
- `onLogScaleChanged(boolean)`: commuta l'escala de l'eix Y entre lineal i logarítmica.

Tots els paràmetres són dades primitives o enums. No hi ha cap referència a `JButton`, `JTextField` ni cap altre component visual. Aquesta és la clau de la separació MVC.

3.2.2. ControlPanel. Panell que conte els camps d'entrada, els botons d'execució i el camp de previsió. Les responsabilitats són exclusivament de la vista:

- 1) Capturar les accions de l'usuari (clics de botons, canvi de checkbox d'escala logarítmica, canvi de visibilitat d'algorismes).

- 2) Validar l'entrada internament: convertir els camps de text a valors `long/int` i mostrar diàlegs d'error (`JOptionPane`) si els valors són invalids.
- 3) Generar l'array de p valors de n distribuïts uniformement entre 1 i n_{final} amb la fórmula:

$$n_i = \frac{n_{\text{final}} \cdot i}{p}, \quad i = 1, \dots, p \quad (3)$$

on p és el nombre de mostres. Només quan la validació passa satisfactòriament s'emet l'event a través de `ViewListener`.

El panell inclou també una fila de *checkboxes* de visibilitat, una per algorisme, que permeten mostrar o ocultar cada sèrie de la gràfica independentment. Quan l'usuari canvia l'estat d'un checkbox, `ControlPanel` notifica `GraphPanel` directament a través d'una interfície interna `VisibilityListener`, sense passar pel controlador MVC, ja que es tracta d'una decisió purament visual que no afecta les dades del model. En ocultar una sèrie, el rang dels eixos es recalcula automàticament en funció de les sèries visibles.

Quan un algorisme està en execució, el seu botó canvia d'icona (►→ ■) i de color (del color de l'algorisme a vermell), permetent aturar-lo individualment amb un segon clic. La Fig. 1 mostra l'aspecte del panell de controls amb els botons i els *checkboxes* de visibilitat.



Figura 1. Panell de controls: botons d'execució per algorisme, botons globals i fila de *checkboxes* de visibilitat.

3.2.3. GraphPanel. Panell personalitzat que sobreescriu `paintComponent` (`Graphics g`) per dibuixar la gràfica amb l'API `Graphics2D`. Les característiques principals són:

- **Eixos i graella:** eix X per a n , eix Y per al temps, amb marques numèriques i línies de graella de color gris clar.
- **Escala dual:** suporta escala lineal i logarítmica a l'eix Y, commutable en temps real. En escala logarítmica, la posició vertical es calcula com:

$$y = h - h \cdot \frac{\log_{10}(t) - \log_{\min}}{\log_{\max} - \log_{\min}} \quad (4)$$

on h és l'alçada del panell en píxels, t és el temps mesurat, i $\log_{\min}$ i $\log_{\max}$ són els logaritmes del valor mínim i màxim de l'eix Y respectivament. La fracció normalitza t dins el rang $[0, 1]$, i la resta $h - h \cdot (\dots)$ inverteix l'eix perquè els valors més grans quedin a la part superior.

- **Sèries de dades:** dibuixa punts per a cada mesura de l'algorisme amb el seu color assignat i un traçat que els connecta.
- **Llegenda:** rectangle semitransparent amb fons blanc i a la cantonada superior esquerra.

L'escala logarítmica és especialment útil quan es comparen algorismes amb ordres de magnitud molt diferents (per exemple, $O(n)$ vs. $O(n^3)$), ja que en escala logarítmica les funcions potencials n^k es visualitzen com rectes amb pendent k .

3.2.4. ResultsPanel. Panell dividit verticalment en dues parts mitjançant un `JSplitPane`:

- Una `JTable` no editable que mostra totes les mesures amb quatre columnes: algorisme, n , temps en ms i constant c en notació científica.
- Una `JTextArea` de només lectura que mostra les constants multiplicatives mitjanes i les previsions sol·licitades per l'usuari.

La Fig. 2 mostra l'aspecte del panell de resultats amb dades reals d'una execució.

Resultats			
Algorisme	n	Temps (ms)	Constant (ns...)
Lineal O(n)	10000	0.0027	2.7080e-01
Lineal O(n)	20000	0.0052	2.5835e-01
Lineal O(n)	30000	0.0078	2.5973e-01
Lineal O(n)	40000	0.0104	2.6043e-01
Lineal O(n)	50000	0.0129	2.5750e-01
Lineal O(n)	60000	0.0154	2.5625e-01
Lineal O(n)	70000	0.0180	2.5714e-01
Lineal O(n)	80000	0.0205	2.5678e-01
Lineal O(n)	90000	0.0232	2.5741e-01
Lineal O(n)	100000	0.0257	2.5708e-01
N-Log N O(n·log n)	10000	0.0947	7.1276e-01
N-Log N O(n·log n)	20000	0.2017	7.0574e-01
N-Log N O(n·log n)	30000	0.3031	6.7938e-01
N-Log N O(n·log n)	40000	0.2697	4.4105e-01
N-Log N O(n·log n)	50000	0.2424	3.1055e-01
N-Log N O(n·log n)	60000	0.3059	3.2122e-01
N-Log N O(n·log n)	70000	0.3648	3.2382e-01
N-Log N O(n·log n)	80000	0.4168	3.1990e-01
N-Log N O(n·log n)	90000	0.5032	3.3971e-01
N-Log N O(n·log n)	100000	0.5605	3.3745e-01
Quadràtic O(n²)	10000	23.7645	2.3765e-01
Quadràtic O(n²)	20000	93.7457	2.3436e-01
Quadràtic O(n²)	30000	208.0263	2.3114e-01

Constants i Previsions	
Constants multiplicatives mitjanes: $c = T(n) / f(n)$	
Lineal	$O(n) : c = 2.5915e-01 \text{ ns}$
N·Log N	$O(n \cdot \log n) : c = 4.4916e-01 \text{ ns}$
Quadràtic	$O(n^2) : c = 2.4148e-01 \text{ ns}$
Cúbic	$O(n^3) : c = 2.3956e-01 \text{ ns}$
--- Previsions per $n = 50,000$ ---	
Lineal	: 0.0130 ms
N·Log N	: 0.3506 ms
Quadràtic	: 603.6883 ms
Cúbic	: 8.32 hores

Figura 2. Panell de resultats: taula de mesures individuals (dalt) i àrea de constants multiplicatives mitjanes i previsions (baix).

3.2.5. MainView. `JFrame` principal que compon els tres sub-panells en un `BorderLayout`. Es registra com a observadora del model amb una lambda:

```
1 model.addListener(() -> {
2     graphPanel.repaint();
3     resultsPanel.updateResults();
4 });
```

Listing 1. Registre com a observadora del model.

L'API pública de `MainView` consta exclusivament de mètodes que operen amb dades:

- `addViewListener(ViewListener)`: registra el controlador com a receptor d'events de la vista.
- `setAlgorithmRunning(AlgorithmType, boolean)`: actualitza l'estat visual del botó d'un algorisme (en execució o aturat).
- `setLogScale(boolean)`: activa o desactiva l'escala logarítmica a la gràfica.
- `showPredictions(long, Map)`: mostra les prediccions de temps calculades pel model per a una mida n donada.

Cap mètode retorna ni rep un component Swing. Aquesta restricció és fonamental per a la separació MVC.

3.3. Capa del Controlador

La classe `Controller` implementa `ViewListener` i gestiona el cicle de vida dels algorismes. Quan l'usuari inicia una execució, el controlador crea un `SwingWorker` que separa el treball en tres fases: `doInBackground()` executa l'algorisme en un fil de fons; `process()` rep els resultats parcials al fil EDT i els afegeix al model; i `done()` actualitza l'estat visual quan la tasca acaba o es cancel·la.

Dins de `doInBackground()`, la mesura de cada punt n segueix dos passos. Primer, es realitza un *escalfament* (warm-up) executant l'algorisme una vegada amb una mida fixa petita abans d'entrar al bucle de mesures. Això força la compilació JIT de la JVM sobre el mètode, evitant que la primera mesura real quedi distorsionada pel cost de compilació. Com que la compilació JIT opera a nivell de mètode i no depèn de la mida de l'entrada, un únic escalfament previ és suficient per a tota la sèrie. Segon, cada valor de n s'executa una única vegada i el temps obtingut es publica directament com a mesura.

El controlador manté un `EnumMap` de `SwingWorker` per poder cancel·lar cada algorisme individualment.

3.4. Esquema general del model MVC implementat

La Fig. 3 mostra el diagrama de classes de l'aplicació amb les relacions entre les tres capes.

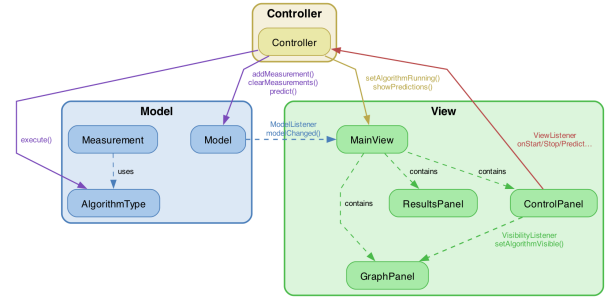


Figura 3. Diagrama de classes i relacions del patró MVC implementat.

4. Càlcul del Cost Asimptòtic per la Instrucció crítica

La instrucció crítica de cada algorisme és l'operació dins el bucle més intern que s'executa el major nombre de vegades. El nombre total d'execucions d'aquesta instrucció determina $f(n)$.

4.1. Algorisme Lineal — $O(n)$

```
1 for (long i = 0; i < n; i++) {
2     sum += i * i;
3 }
```

Listing 2. Algorisme lineal.

La instrucció crítica `sum += i * i` s'executa exactament n vegades. No hi ha bucles imbricats, per tant:

$$f(n) = n \implies T(n) \in O(n) \quad (5)$$

4.2. Algorisme Quasilineal — $O(n \cdot \log n)$

```
1 for (long i = 0; i < n; i++) {
2     for (long j = 1; j < n; j *= 2) {
3         sum += i ^ j;
4     }
5 }
```

Listing 3. Algorisme $O(n \cdot \log n)$.

El bucle extern itera n vegades. El bucle intern comença a $j = 1$ i dobla j a cada iteració (`j *= 2`), de manera que s'executa exactament $\lfloor \log_2 n \rfloor$ vegades per cada i . Aquesta estructura reflecteix directament la divisió successiva per 2 pròpia dels algorismes quasilineals reals (com ara Merge-sort). La instrucció crítica `sum += i ^ j` s'executa:

$$f(n) = n \cdot \lfloor \log_2 n \rfloor \implies T(n) \in O(n \log n) \quad (6)$$

4.3. Algorisme Quadràtic — $O(n^2)$

```

1 for (long i = 0; i < n; i++) {
2     for (long j = 0; j < n; j++) {
3         sum += i * j;
4     }
5 }

```

Listing 4. Algorisme quadràtic.

Dos bucles imbricats, cadascun de n iteracions. La instrucció crítica `sum += i * j` s'executa:

$$f(n) = n \times n = n^2 \implies T(n) \in O(n^2) \quad (7)$$

4.4. Algorisme Cúbic — $O(n^3)$

```

1 for (long i = 0; i < n; i++) {
2     for (long j = 0; j < n; j++) {
3         for (long k = 0; k < n; k++) {
4             sum += (i*j) + k;
5         }
6     }
7 }

```

Listing 5. Algorisme cúbic.

Tres bucles imbricats, cadascun de n iteracions. La instrucció crítica `sum += (i * j) + k` s'executa:

$$f(n) = n^3 \implies T(n) \in O(n^3) \quad (8)$$

4.5. Resum Comparatiu

La Taula 2 resumeix la instrucció crítica, el nombre d'execucions i la complexitat de cada algorisme.

Taula 2. INSTRUCCIÓ CRÍTICA I COMPLEXITAT DE CADA ALGORISME.

Algorisme	Instrucció crítica	Execucions	O
Lineal	<code>sum += i*i</code>	n	n
N Log N	<code>sum += i^j</code>	$n \lfloor \log_2 n \rfloor$	$n \log n$
Quadràtic	<code>sum += i*j</code>	n^2	n^2
Cúbic	<code>sum += (i*j) + k</code>	n^3	n^3

4.6. Constant Multiplicativa i Predicció

Per a cada mesura individual, la constant multiplicativa es calcula segons l'equació (2). La constant mitjana $\bar{c}$ s'obté com la mitjana aritmètica de totes les constants individuals d'un algorisme:

$$\bar{c} = \frac{1}{m} \sum_{k=1}^m c_k = \frac{1}{m} \sum_{k=1}^m \frac{T(n_k)}{f(n_k)} \quad (9)$$

on m és el nombre de mesures realitzades. Aquesta constant reflecteix el cost temporal per unitat d'operació bàsica a la màquina concreta.

Per fer una predicció del temps d'execució per a un valor de n no mesurat, s'utilitza la fórmula:

$$\hat{T}(n) = \bar{c} \cdot f(n) \quad (10)$$

Per exemple, si tenim $\bar{c} = 2.5$ ns per a l'algorisme quadràtic i volem predir $n = 100\,000$:

$$\hat{T}(100\,000) = 2.5 \times (100\,000)^2 = 2.5 \times 10^{10} \text{ ns} = 25 \text{ s} \quad (11)$$

Aquesta capacitat de predicció és un dels avantatges clau de l'anàlisi asimptòtic: amb unes poques mesures per a valors petits de n , es pot estimar el temps d'execució per a valors molt més grans sense necessitat d'executar l'algorisme.

5. Resultats i Discussió

L'aplicació implementada permet verificar experimentalment que els temps d'execució mesurats segueixen les funcions de cost teòric. A la gràfica, s'observa clarament com:

- La sèrie lineal $O(n)$ creix de forma gairebé imperceptible en comparació amb les altres sèries.
- La sèrie $O(n \cdot \log n)$ creix lleugerament més que la lineal, però de forma moderada i controlada.
- La sèrie quadràtica $O(n^2)$ creix de forma parabòlica, essent significativament més costosa per a valors grans de n .
- La sèrie cúbica $O(n^3)$ domina clarament totes les altres, fent-se impracticable per a valors de n grans.

La Fig. 4 i la Fig. 5 il·lustren clarament la necessitat de l'escala logarítmica. En escala lineal, la sèrie cúbica domina de tal manera que les tres sèries inferiors queden comprimides contra l'eix X i resulten indistingibles. En escala logarítmica, les quatre sèries se separen visualment i les funcions potencials n^k es visualitzen com a rectes amb pendent proporcional a k , facilitant la verificació visual de la complexitat.

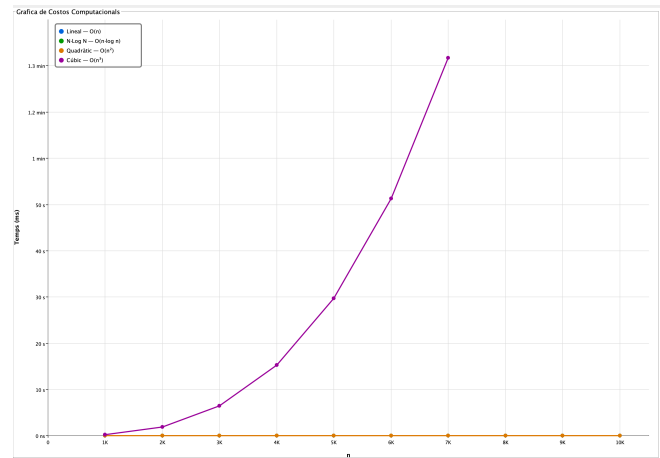


Figura 4. Gràfica en escala lineal: les sèries $O(n)$, $O(n \log n)$ i $O(n^2)$ queden comprimides contra l'eix X per la dominància de $O(n^3)$.

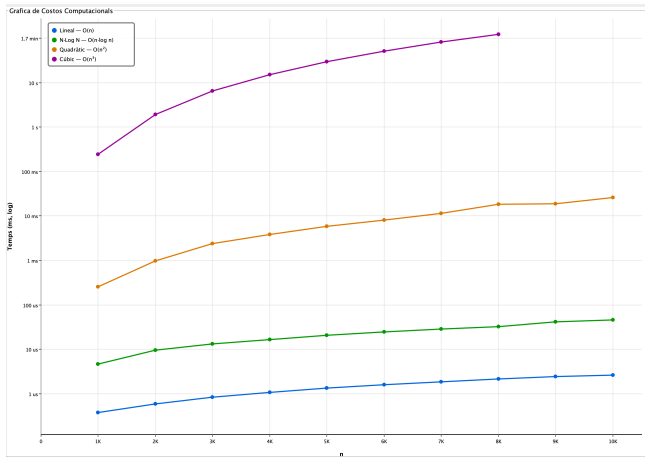


Figura 5. Gràfica en escala logarítmica: les quatre sèries se separen clarament i les pendent reflecteixen els ordres de creixement 1, ≈ 1 , 2 i 3 respectivament.

L'ús de l'escala logarítmica a l'eix Y permet comparar totes quatre sèries simultàniament sense que les més ràpides quedin comprimides contra l'eix X. En escala logarítmica, les funcions potencials n^k es visualitzen com a rectes amb pendent proporcional a k , facilitant la verificació visual de la complexitat.

Les constants multiplicatives calculades es mantenen raonablement estables per a un algorisme donat, la qual cosa confirma que el model asimptòtic $T(n) = c \cdot f(n)$ és una bona aproximació. Petites variacions s'atribueixen a:

- Efectes de la jerarquia de memòria: per a valors grans de n , les dades poden excedir la cache, augmentant la latència.
- La compilació JIT de la JVM: el compilador optimitza el codi progressivament durant l'execució, la qual cosa pot fer que les primeres mesures siguin més lentes. L'escalfament previ mitiga parcialment aquest efecte.
- La planificació del sistema operatiu: altres processos poden competir pel temps de CPU.

La funcionalitat de predicció ha mostrat resultats coherents: les prediccions per a valors de n dins el rang mesurat concorden amb les mesures reals, i les prediccions per a valors més grans segueixen l'extrapolació esperada.

Respecte al disseny del programari, l'estricta separació MVC amb comunicació per esdeveniments ha permès:

- Desenvolupar i provar la lògica del model independentment de la vista.
- Modificar la interfície gràfica (per exemple, afegir l'escala logarítmica o millorar el format de la gràfica) sense alterar el controlador ni el model.
- Garantir que l'aplicació roman responsiva durant l'execució d'algorismes lents, gràcies a l'ús de `SwingWorker` que executa els algorismes en fils de fons separats del fil EDT.
- Cancel·lar algorismes individualment sense afectar els altres que s'estan executant concurrentment.

6. Conclusions

Aquesta pràctica ha permès aplicar de forma conjunta els conceptes d'anàlisi asimptòtic i de disseny de programari amb el patró MVC.

Des del punt de vista teòric, s'ha verificat experimentalment que els temps d'execució dels quatre algorismes implementats corresponen a les seves complexitats teòriques. La identificació de la instrucció crítica de cada algorisme — l'operació del bucle més intern — i el càlcul de la constant multiplicativa han permès fer prediccions fiables del temps d'execució per a mides d'entrada arbitràries. L'experiment confirma que el model $T(n) = c \cdot f(n)$ és una aproximació vàlida quan n és prou gran i les condicions d'execució són estables.

Des del punt de vista del disseny de programari, s'ha aconseguit una implementació estricta del patró MVC on:

- 1) La vista **no exposa cap component visual** al controlador; tota interacció és a través d'events semàntics.
- 2) La interfície `ViewListener` defineix set events amb paràmetres de dades (enums, arrays de `long`, booleans), mai objectes `Swing`.
- 3) El model notifica canvis a través de `ModelListener` sense conèixer qui l'observa.
- 4) El controlador opera exclusivament amb dades, delegant la presentació visual a la vista i la lògica de negoci al model.
- 5) La concurrència es gestiona amb `SwingWorker`, garantint una interfície responsiva i una cancel·lació segura.

El patró per esdeveniments ha demostrat ser essencial per mantenir el desacoblament entre capes, permetent que cada component evolucioni de forma independent. Aquesta separació no és una mera formalitat acadèmica: facilita el manteniment, la testabilitat i l'extensibilitat del programari. Per exemple, es podria substituir la vista `Swing` per una interfície `JavaFX` o web sense modificar ni una línia del model o del controlador.

Com a treball futur, es podrien afegir més algorismes (per exemple, $O(2^n)$ o $O(n!)$), implementar l'exportació de dades a CSV, permetre a l'usuari definir algorismes personalitzats, o afegir més estratègies de mesura com la repetició múltiple amb mitjana per reduir la variància experimental.

Referències

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [2] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal, *Pattern-Oriented Software Architecture: A System of Patterns*. Chichester, U.K.: Wiley, 1996.
- [3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2011.

- [5] J. Elliott, R. Eckstein, M. Loy, D. Wood, and B. Cole, *Java Swing*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2002.
- [6] Oracle Corporation, "The Java Tutorials — Concurrency," [Online]. Available: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- [7] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.